AOK–MCP

Admissibility-Governed Tool Protocols Over Open Model Interfaces

The proposal is evaluated under explicit admissibility law, executed only through a typed effect membrane, and sealed with a receipt that binds execution to lineage.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Open tool protocols such as Model Context Protocol (MCP) standardize how models request external actions. They define schemas, transport formats, and invocation patterns. They do not define under what law a tool call may count as having legitimately occurred. Protocol governs syntax and routing; it does not govern admissibility, refusal, replay, or proof.

This paper introduces **AOK–MCP**, a sovereignty-preserving interpretation of open tool protocols. Within AOK–MCP, a tool call is not an instruction but a **proposed semantic mutation**. The proposal is evaluated under explicit admissibility law, executed only through a typed effect membrane, and sealed with a receipt that binds execution to lineage. Refusal is first-class and receipted. Absence of a conforming receipt collapses the action to an epistemic non-event.

We formalize AOK–MCP as a state transition discipline over content-addressed law, schema, and receipt artifacts, define an explicit Trusted Computing Base (TCB), parameterize cryptographic assumptions, and state and prove uniqueness and replay determinism properties. We clarify receipt schema evolution, specify determinism obligations and enforcement mechanisms, scope idempotency guarantees to semantic state advancement, and demonstrate a concrete policy-drift failure in naïve MCP that AOK–MCP structurally prevents. Performance measurements are reported with explicit methodology. AOK–MCP leaves protocol wire formats unchanged while relocating authority upstream of execution.

---

1. Motivation: The Admissibility Gap

Modern tool-calling ecosystems increasingly mediate:

- financial transactions,

- infrastructure orchestration,
- data modification across compliance-bound systems,
- safety-critical actions triggered by model outputs.

Observed failure modes include:

- silent double execution under retries,
- audit reinterpretation after policy updates (policy drift),
- logging without cryptographic binding to execution context,
- backend heterogeneity leading to ambiguous replay,
- ambiguous refusal semantics (policy denial vs substrate failure).

Open tool protocols (including MCP) specify transport and invocation. They do not bind execution to versioned admissibility law, nor do they require receipt-bound state advancement.

AOK–MCP addresses this gap by binding admissibility, execution, refusal, and proof into a single state transition discipline.

---

2. Cryptographic and Computational Assumptions

We assume probabilistic polynomial-time adversaries.

C1 (Collision Resistance). H is collision-resistant against polynomial-time adversaries.

C2 (Second-Preimage Resistance). Given x, it is infeasible for a polynomial-time adversary to find $x' \neq x$ such that $H(x') = H(x)$.

Guarantees are conditional on C1–C2 and integrity of the Trusted Computing Base.

---

3. Content-Addressed Artifacts

AOK–MCP treats laws, schemas, and receipts as **content-addressed artifacts**.

Content-addressing means artifacts are identified by their cryptographic hash:

$$artifact_id = h(artifact)$$

Any change to the artifact (law, schema, or receipt) produces a different hash, detectable during validation. Content-addressing ensures immutability by identity and verifiability by recomputation of the hash.

---

4. Formal Model

4.1 State Space

Define:

$$S = (Lineage, LawStore, SchemaStore, ReceiptStore)$$

Where:

- `Lineage` is an append-only ordered sequence of receipt hashes.
- `LawStore : Hash \rightarrow LawArtifact`.
- `SchemaStore : Hash \rightarrow SchemaArtifact`.
- `ReceiptStore : Hash \ rightarrow ReceiptArtifact`.

This specification assumes a **single-host lineage authority** for ordering. Distributed consensus for multi-host deployments is out of scope.

---

5. Canonicalization and Step Identity

Let:

- `canon(x)` be injective canonical serialization.
- $h(x) = H(canon(x))$.

A Step is:

$$Step = (type, target, payload, effect_class, law_hash)$$

with identity:

$$step_hash = h(Step)$$

Lemma (Step Hash Uniqueness). Under injective canonicalization and C1–C2, distinct canonical Steps produce distinct `step_hash` values with overwhelming probability.

---

6. Admissibility Law

Each law artifact defines a pure function:

$$L : Step \rightarrow \{ALLOW, DENY\}$$

Law Determinism Constraint.

Law artifacts must:

- be referentially transparent,
- depend only on canonical Step and law artifact content,
- terminate within declared bounds.

Steps bind the specific `law_hash` under which admissibility is evaluated.

---

7. Trusted Computing Base

7.1 Adversarial Components

- Model output
- Transport layer
- Backend services
- Network

7.2 Trusted Components

- Canonicalization implementation
- Hash implementation
- Law evaluation engine
- Receipt validation logic
- DomainEffects implementation
- Lineage append operation (`apply`)

7.3 TCB Minimization

The TCB can be minimized by:

- isolating DomainEffects as a narrowly scoped module,
- implementing canonicalization and hashing via well-audited libraries,
- treating LawStore and SchemaStore as immutable artifact registries,
- using formal interfaces for effect_class lowering.

AOK–MCP does not claim formal verification of the TCB; implementation correctness is assumed.

---

8. DomainEffects and Backend Transcript

DomainEffects is the execution membrane and part of the TCB.

8.1 Backend Response Transcript

A **backend response transcript** consists of:

- canonicalized request payload,
- HTTP or protocol status code,
- canonicalized response body,
- backend-generated identifiers (if any),
- headers required for semantic interpretation.

Timing data, transport-level nondeterminism, and non-semantic metadata are excluded.

Determinism is defined over canonical Step plus backend response transcript.

8.2 Determinism Obligation

Given identical canonical Step and identical backend response transcript, DomainEffects must produce:

- identical canonical Receipt, or
- refusal.

Nondeterministic behavior violates TCB assumptions.

Enforcement occurs via specification obligations and replay-based regression testing.

---

9. Receipt Structure and Schema Evolution

Receipt: Receipt(step_hash, law_hash, tool_id, effect_class, backend_id, outcome, payload_digest, receipt_schema_hash, sequence_coordinate)

Receipt hash:

$$receipt_hash = h(Receipt)$$

Receipt hashes commit to the entire canonical receipt object.

9.1 Schema Evolution

When receipt schema evolves:

- new receipts bind `receipt_schema_hash = h(Schema_v2)`,
- historical receipts remain valid under their original schema,
- replay validates each receipt using its bound schema artifact.

---

10. State Evolution

Define:

$$apply(Lineage, Receipt) = Lineage \mid [receipt_hash]$$

The only admissible state mutation is via `apply`. Lineage is append-only.

---

11. Theorem 1: Unique Step-to-Receipt Binding

Under injective canonicalization, C1–C2, and full receipt commitment:

No two distinct Steps can produce lineage-equivalent receipts.

Proof follows from:

- Step hash uniqueness,
- receipt hash commitment to full receipt,
- binding of `step_hash`, `law_hash`, and `receipt_schema_hash`.

□

---

12. Proposition 1: Replay Determinism

Given:

- identical ordered receipt hashes in Lineage,
- availability and integrity of content-addressed artifacts referenced by those hashes,
- deterministic law evaluation and receipt validation,

replay produces identical lineage state.

Replay depends on retrievability and integrity of the artifacts addressed by receipt-bound hashes, not on identical store layouts.

□

---

13. Idempotent Effect Classes

For declared idempotent classes:

- idempotency key derived from `step_hash`,
- backend identifier bound in receipt.

AOK–MCP prevents silent semantic duplication without distinct receipts. Physical duplicate execution may occur without backend cooperation but does not advance lineage state.

---

14. Policy Drift and Non-Retroactivity

Policy v1 allows transfer <= $10k. Step binds `law_hash = h(L_v1)`. Transfer executes. Policy v2 reduces limit to $5k.

In naïve MCP, historical admissibility may be reinterpreted.

In AOK–MCP:

- Receipt binds `law_hash`.
- Replay retrieves historical law artifact.
- Admissibility is evaluated under original law.

AOK–MCP does not allow retroactive policy changes to affect historical admissibility. Once a law artifact is bound to a Step, it cannot be reinterpreted under newer policy without breaking cryptographic assumptions or compromising the TCB.

---

15. Performance Evaluation

Prototype:

- Rust 1.74
- 8-core CPU, 32GB RAM
- SHA-256
- Deterministic JSON
- 10,000 Steps, 512-byte payload

Baseline:

- Mean: 1.83 ms
- standard deviation: 0.14 ms

AOK–MCP:

- Mean: 2.02 ms
- standard deviation: 0.16 ms

Overhead approximately 10.4%.

Replay validation:

- 10,000 receipts validated in 18.7 ms.

---

16. Comparison

- **Property:** Admissibility Gate
  Naïve MCP: No
  Event Sourcing: No
  AOK–MCP: Yes
- **Property:** Law Hash Binding
  Naïve MCP: No
  Event Sourcing: No
  AOK–MCP: Yes
- **Property:** Schema Hash Binding
  Naïve MCP: No
  Event Sourcing: No
  AOK–MCP: Yes
- **Property:** Receipt-bound State
  Naïve MCP: No
  Event Sourcing: Partial
  AOK–MCP: Yes
- **Property:** Refusal as Artifact
  Naïve MCP: No
  Event Sourcing: Partial
  AOK–MCP: Yes
- **Property:** Deterministic Replay
  Naïve MCP: No
  Event Sourcing: Partial
  AOK–MCP: Yes
- **Property:** Idempotent Effect Support
  Naïve MCP: No
  Event Sourcing: No
  AOK–MCP: Yes
- **Property:** Policy Evolution Safety
  Naïve MCP: No
  Event Sourcing: No
  AOK–MCP: Yes

---

17. Related Work

Open tool protocols (e.g., MCP specification) standardize invocation but do not cryptographically bind admissibility decisions to execution artifacts.

Event sourcing records events but does not bind them to versioned admissibility law or require receipts as a condition of state advancement.

Capability systems constrain authority but do not require receipt production for semantic lineage evolution.

Zero-trust and policy engines focus on access control, not replay determinism or version-bound law receipts.

AOK–MCP contributes a content-addressed admissibility layer over open tool protocols.

---

18. Future Work

Future directions include:

- distributed lineage consensus,
- formal verification of DomainEffects,
- minimization and isolation of the TCB,
- integration with AI safety frameworks (constitutional AI, debate systems),
- hardware-backed artifact integrity guarantees.

---

19. Summary

Open tool protocols define how models call tools but leave admissibility ungoverned. This creates an admissibility gap: execution can occur without version-bound law, refusal artifacts, or replay guarantees.

AOK–MCP closes this gap by combining:

- content-addressed law artifacts,
- deterministic admissibility evaluation,
- a typed execution membrane,
- receipt-bound state advancement.

The key innovation is binding tool execution to versioned admissibility law via cryptographic receipts. This prevents retroactive reinterpretation, enforces semantic determinism, and enables auditability across policy evolution.

AOK–MCP does not replace existing protocols; it reorders authority within them.

Corpus-Wide Reference Positioning

This paper argues that open tool protocols such as Model Context Protocol (MCP) standardize how models request external actions. Its nearest external comparison fields are ai, formal, security. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or

that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Retrieval, tool use, and policy layers can reduce unsupported output without eliminating model error, source error, omission, bias, or the environmental and institutional costs of scale. Formal assurance remains relative to the encoded model, trusted base, and refinement chain; proof does not remove specification error or unmodeled environmental behavior. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS 2020.* arXiv:2005.11401. https://arxiv.org/abs/2005.11401 **Relationship:** Predecessor retrieval architecture; comparison centers on provenance, admission, and fail-closed constraints not supplied by retrieval alone.

[**E2**] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023.* arXiv:2210.03629. https://arxiv.org/abs/2210.03629 **Relationship:** Predecessor agent pattern for interleaving reasoning and action; used to distinguish action traces from governed effects.

[**E3**] Elham Tabassi. *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, 2023. DOI: 10.6028/NIST.AI.100-1. https://doi.org/10.6028/NIST.AI.100-1 **Relationship:** Adjacent risk-management framework used to test whether the paper operationalizes governance beyond voluntary process guidance.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence.* OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstruments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] Emily M. Bender et al. "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" *FAccT '21*, 2021. DOI: 10.1145/3442188.3445922. https://doi.org/10.1145/3442188.3445922 **Relationship:** Counterpressure concerning scale, environmental cost, documentation, and social risk; included to prevent governance proposals from becoming capability-only narratives.

[**E6**] George C. Necula. "Proof-Carrying Code." *POPL '97*, 1997, pp. 106-119. DOI: 10.1145/263699.263712. https://doi.org/10.1145/263699.263712 **Relationship:** Predecessor for binding machine-checkable evidence to executable artifacts; the present paper may demand broader semantic or constitutional obligations.

[**E7**] Xavier Leroy. "Formal Verification of a Realistic Compiler." *Communications of the ACM* 52(7), 2009, pp. 107-115. DOI: 10.1145/1538788.1538814. https://doi.org/10.1145/1538788.1538814 **Relationship:** Predecessor verified-compilation result used to distinguish proven translation preservation from broader governance claims.

Ableman corpus

[**A1**] Adam Ableman. *CARA: Constrained Artificial Reasoning Architecture.* Ableman Corpus current edition; work identity `PAPER.PUB.18099715`;
source SHA-256
`F8F74F0CD58F5414D9A29EBE958E49751892D07A028CB51197F8201B18AAB40E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *A Constitution for the Context Window.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-CF26BA0E123F472F`;
source SHA-256
`60395402368E146F115CCE53D0E5A15B8FB23C90ED5B24CA92AB8515B2009965`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Governed Context & Memory (GCM).* Ableman Corpus current edition; work identity `PAPER.PUB.18633797`;
source SHA-256

3F17ACD8AB9C6DC66C23D22098628C70042854B198C3ABFCBFA3EC7DC27B9C08.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Retrieval-Augmented Reasoning under Constitutional Constraints*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-384489D4604BEC4D;
source SHA-256
133DA35C0F27F35BAA1F53DFEE415068AD2D24FF713F8CDAFA2FDC8F748AC15C.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Split-Authority AI Coordination*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-35D1BAECD5CB17AD;
source SHA-256
06FB6133170879FA8308684F0DA33F5813EAF3B5488291C2D7D234BEB6FDFEC2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *MCIR: A Canonical Intermediate Representation for Governed Transformation*. Ableman Corpus current edition; work identity PAPER.PUB.21095303;
source SHA-256
EA129ECBA95F7B07CAA7B33A54F91F52913C94A14DA227EB9A06CFD083E4E5BF.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governed Semantic Compilation*. Ableman Corpus current edition; work identity PAPER.PUB.18453681;
source SHA-256
5FF6C636550B04DB0E2F6F94C379BB75AA0BD8781E19B047D7D3BB6D49071AC2.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.18617635